

PROYECTO 3

Sistema de Resolución de Conflictos en una Negociación Automatizada

Grafos dirigidos · Árboles de decisión · Autómatas finitos deterministas



Modelar una negociación entre varios participantes

1

Decisiones sucesivas

Cada participante elige, etapa por etapa, entre Cooperar, Competir o Ceder.

2

Intereses propios

Cada opción tiene un peso distinto para cada participante (0 a 10).

3

Efectos encadenados

La decisión de uno influye en las opciones futuras de los demás.

4

Resultado a evaluar

El sistema debe decir si se llega a un acuerdo estable o a un conflicto.

Tres modelos discretos trabajando juntos

1

Grafos

Modelan quién influye sobre quién:
matriz de adyacencia, grados,
conexidad y ciclos.

2

Árbol de decisión

Modela todas las combinaciones
posibles de decisiones y busca
la mejor estrategia conjunta.

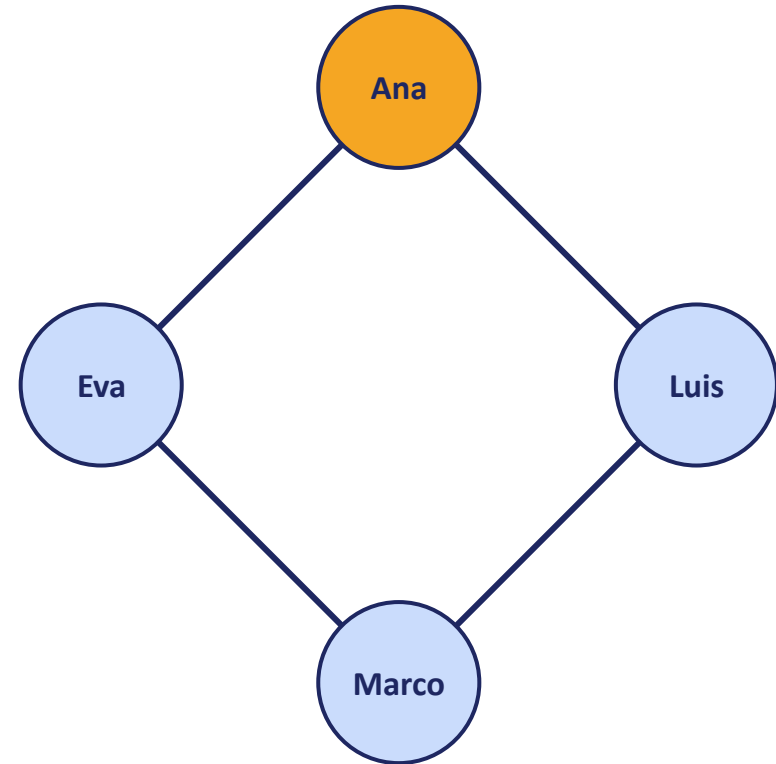
3

Autómata finito

Modela cómo evoluciona el estado
de la negociación etapa a etapa
hasta un acuerdo o conflicto.

Grafo dirigido de influencia

- Cada participante es un vértice.
- Cada arista dirigida $i \rightarrow j$ significa "la decisión de i influye sobre j ".
- Se representa con una matriz de adyacencia: $\text{grafo}[i][j] = 1$ si existe la arista.
- Con el grado de entrada/salida medimos cuánto influye y es influido cada participante.



*Ana → Luis → Marco → Eva → Ana
(circuito cerrado tipo anillo)*

Construcción de la matriz de adyacencia

```
void construirGrafoInfluencia(int
grafo[][MAX_PARTICIPANTES],
                             int numParticipantes) {
    for (int i = 0; i < numParticipantes; i++)
        for (int j = 0; j < numParticipantes; j++)
            grafo[i][j] = 0;

    for (int i = 0; i < numParticipantes; i++) {
        int siguiente = (i + 1) % numParticipantes;
        grafo[i][siguiente] = 1; // arista i -> siguiente
    }
}
```

- Primero limpia la matriz (todo en 0: sin aristas).
- El módulo % conecta al último participante de vuelta con el primero, cerrando el circuito en forma de anillo.
- `gradoSalida()` cuenta 1's en una fila; `gradoEntrada()` los cuenta en una columna.

	A	L	M	E
A	0	1	0	0
L	0	0	1	0
M	0	0	0	1
E	1	0	0	0

¿El grafo conecta a todos? ¿Cierra el circuito?

```
// esConexo(): BFS manual con cola propia (sin <queue>)
cola[finCola++] = 0; visitado[0] = true;
while (inicioCola < finCola) {
    int actual = cola[inicioCola++];
    for (vecino = 0; vecino < n; vecino++) {
        bool hay = grafo[actual][vecino]==1 ||
                  grafo[vecino][actual]==1;
        if (hay && !visitado[vecino]) {
            visitado[vecino] = true;
            cola[finCola++] = vecino;
        }
    }
}
return visitados == numParticipantes;
```

BFS (recorrido en anchura)

Visita vértice por vértice usando una cola: se marca cada uno como visitado y se revisan sus vecinos en ambos sentidos, solo para medir conexidad.

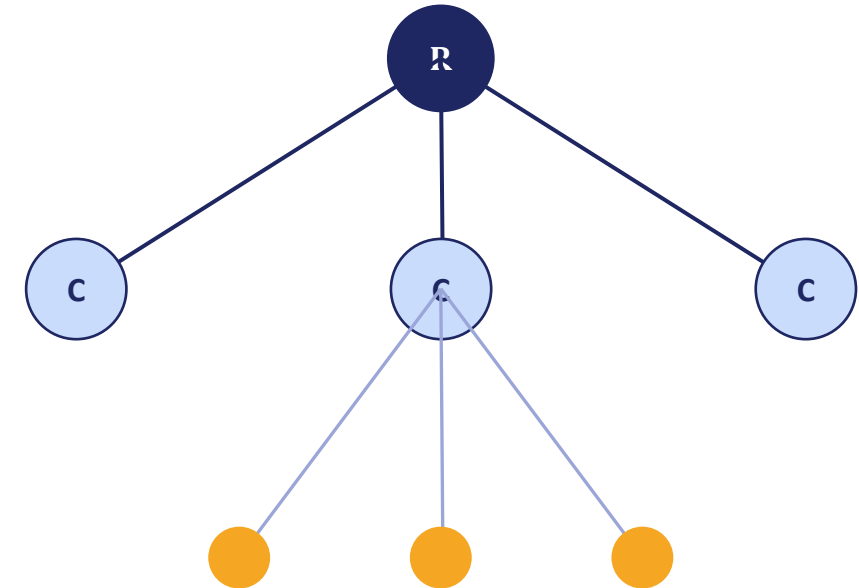
tieneCicloCompleto()

Sigue las flechas salientes desde el vértice 0. Si tras numParticipantes pasos regresa exactamente al inicio sin repetir vértices antes de tiempo, el circuito es un ciclo hamiltoniano.

✓ Conexo y con ciclo hamiltoniano → la influencia recorre y regresa a todos los participantes.

Árbol de decisión: todas las combinaciones posibles

- Cada nivel del árbol = la decisión de un participante.
- Cada nodo tiene 3 hijos: Cooperar, Competir, Ceder.
- Cada hoja = una combinación completa de decisiones, con su puntaje conjunto acumulado.
- Se construye con memoria dinámica: new crea cada nodo; delete la libera al final.



Nivel 1: decisión del participante 1 Nivel 2: decisión del participante 2 ...

Con 3 opciones y n participantes: 3^n hojas posibles

Construcción recursiva del árbol de juego

```
NodoArbol* construirArbolDecision(
    Participante participantes[], int numParticipantes,
    int participanteActual, int opcionQueLlego,
    int puntajeAcumulado) {

    NodoArbol* nodo = crearNodo(participanteActual,
                                opcionQueLlego, puntajeAcumulado);

    if (participanteActual >= numParticipantes)
        return nodo;                // caso base: hoja

    for (int o = 0; o < NUM_OPCIONES; o++) {
        int nuevoPuntaje = puntajeAcumulado +
            participantes[participanteActual].intereses[o];
        nodo->hijos[o] = construirArbolDecision(participantes,
            numParticipantes, participanteActual + 1,
            o, nuevoPuntaje);        // caso recursivo
    }
    return nodo;
}
```

Caso base

Cuando ya se decidió por todos los participantes, el nodo se vuelve hoja y se detiene la recursión.

Caso recursivo

Por cada una de las 3 opciones se suma el interés correspondiente y se crea el hijo llamando de nuevo a la función, avanzando al siguiente participante.

Recorridos y la mejor estrategia conjunta

```
// Preorden: raiz, luego cada hijo (0,1,2)
// Postorden: cada hijo, luego la raiz

void buscarMejorHoja(NodoArbol* nodo, ...,
    int &mejorPuntaje, int mejorRuta[]) {
    bool esHoja = (hijos[0]==nullptr &&
        hijos[1]==nullptr && hijos[2]==nullptr);
    if (esHoja) {
        if (nodo->puntajeAcumulado > mejorPuntaje) {
            mejorPuntaje = nodo->puntajeAcumulado;
            // copiar rutaActual -> mejorRuta
        }
        return;
    }
    for (int i = 0; i < NUM OPCIONES; i++)
        buscarMejorHoja(nodo->hijos[i], ...);
}
```

Teoría de juegos aplicada

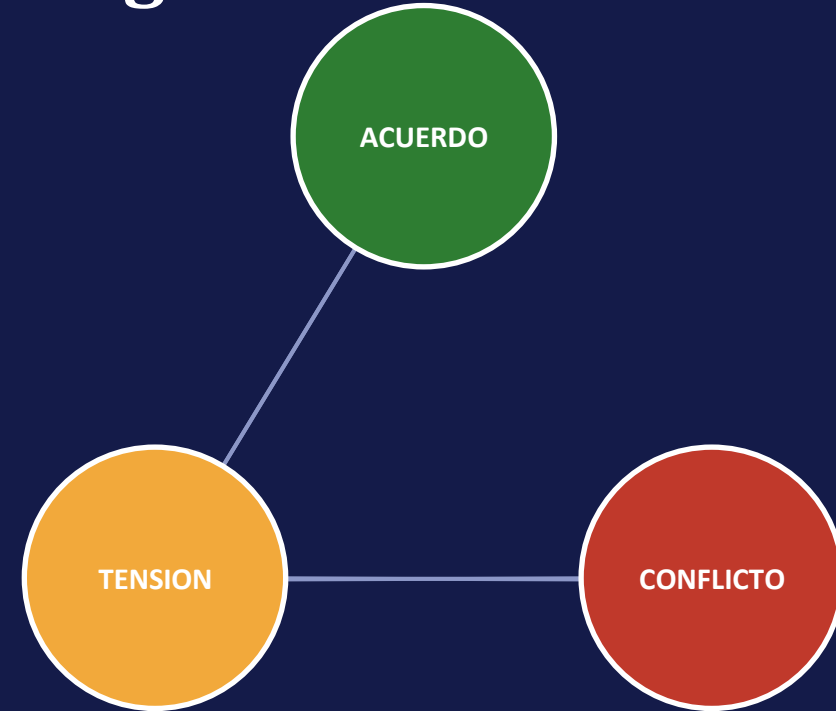
Recorre cada hoja del árbol (cada combinación posible de decisiones) y se queda con la de mayor puntaje conjunto: la mejor estrategia para todos.

mejorPuntaje y mejorRuta[] viajan por referencia (&)

Así se van actualizando durante toda la recursión y conservan su valor final sin necesidad de un return explícito.

Autómata finito determinista de la negociación

- Estados: ACUERDO, TENSION, CONFLICTO.
- Alfabeto: alta / media / baja cooperación en cada etapa.
- Estado inicial: TENSION. Estado de aceptación: ACUERDO.
- Es determinista (AFD): cada par (estado, símbolo) tiene un único resultado.



alta cooperación mejora el estado · baja cooperación lo empeora

Tabla de transición y simulación del AFD

Estado	alta_coop	media_coop	baja_coop
ACUERDO	ACUERDO	TENSION	CONFLICTO
TENSION	ACUERDO	TENSION	CONFLICTO
CONFLICTO	TENSION	CONFLICTO	CONFLICTO

```
int transicion(int estadoActual, int simbolo) {
    if (estadoActual == ESTADO_ACUERDO) {
        if (simbolo == SIMBOLO_ALTA_COOP)
            return ESTADO_ACUERDO;
        if (simbolo == SIMBOLO_MEDIA_COOP)
            return ESTADO_TENSION;
        return ESTADO_CONFLICTO;
    }
    // ... TENSION y CONFLICTO siguen la
    // misma logica con su fila de la tabla
}
```

- Cada etapa cuenta cooperaciones y las clasifica en un símbolo (clasificarSimbolo).
- transicion() aplica la fila/columna correspondiente de la tabla.
- Si el autómata termina en ACUERDO, la cadena de etapas es "aceptada": la negociación converge.

¿Acuerdo estable, parcial o conflicto?

Se mide cuántos participantes cooperaron en la última etapa:

ACUERDO ESTABLE

Todos cooperaron en la etapa final.

ACUERDO PARCIAL

Más de la mitad cooperó, pero persisten diferencias.

CONFLICTO

Predominó Competir o Ceder: los intereses no se alinearon.

```
if (cooperacionesFinales == numParticipantes)
    -> "ACUERDO ESTABLE"
else if (cooperacionesFinales * 2 >= numParticipantes)
    -> "ACUERDO PARCIAL"
else
    -> "CONFLICTO"
```

CONCLUSIÓN

Un sistema, tres modelos, un solo flujo

1

Inicializar participantes

2

Simular la negociación (etapas)

3

Evaluar resultados y estabilidad

4

Construir y analizar el grafo

5

Construir árbol y hallar mejor estrategia

6

Simular el autómata final

Lo que aporta cada tema

Grafos

Explican quién influye sobre quién y si esa influencia llega a todos.

Árboles

Exploran todas las combinaciones y revelan la mejor estrategia conjunta.

Autómatas

Resumen la evolución completa de la negociación en un solo resultado.

Resultado final: el sistema simula, visualiza y justifica si la negociación converge a un acuerdo estable.